Protocols and layers

agreed languages, and why we stack them

Watch the 9-min video

In one line: a protocol is a set of agreed rules so two machines understand each other; the internet splits the work into four layers, application, transport, network and link, each with its own protocol, and each layer adds its own header on the way down and removes it on the way up.

WORDS YOU'LL MEET TODAY

- **Protocol**: agreed rules for talking, so two sides understand each other
- **Layer**: one job in a stack of jobs, with its own protocol
- **Header**: a label a layer adds in front of the data
- **Encapsulation**: wrapping data inside a header, layer by layer
- **Segment**: data plus a TCP header (transport layer)
- **Packet**: a segment plus an IP header (network layer)
- **Frame**: the package for one link: a packet plus a link header and check value
- **MTU**: the biggest packet one link carries: 1,500 bytes on Ethernet

1 The question: how do strangers understand each other?

You open a shopping app and tap Buy. Your phone was made by one company, the shop's server by another, and your order crosses Wi-Fi, a fibre cable and routers built by many others. None of them ever met, yet your order arrives and is understood perfectly. Two ideas make that possible: **protocols** and **layers**.

2 A protocol is agreed rules

You already use one every day: a phone call. Someone picks up and says hello, you say who you are, you take turns to talk, then you both say bye and hang up. Nobody wrote these rules down, but both sides follow them. If one side stayed silent, or spoke a language the other didn't know, the call would fail.

Computers need the same, but exact. In **HTTP**, the protocol of the web, the browser sends `GET /shoes`, and the server must answer with a status, like `200 OK`, and then the page. Both follow the same rulebook, so machines from different companies understand each other.



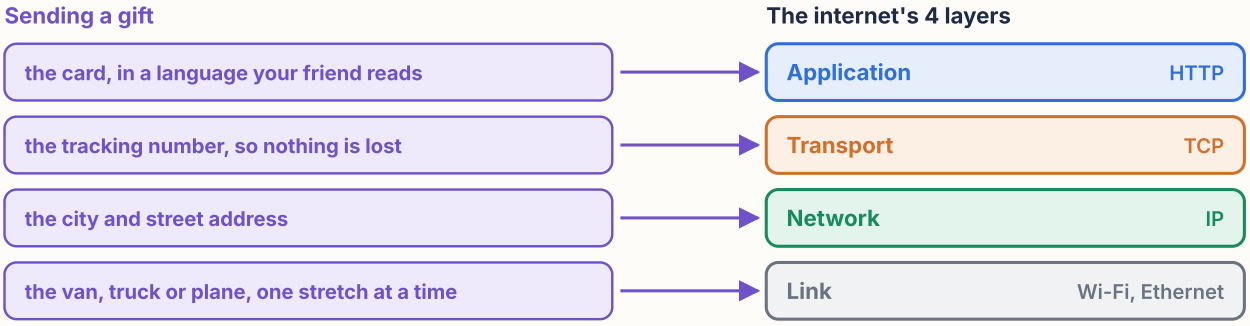
A protocol is agreed rules, so two sides understand each other. If one side breaks them, the conversation fails.

Figure 1. A protocol is a set of agreed rules. A phone call follows unwritten ones; HTTP writes them down exactly, so machines from different companies understand each other.

3 Why one protocol isn't enough: layers

One protocol can't do everything. Someone must find the address, someone must check that nothing was lost, someone must move the bits over Wi-Fi. So the work is split into **layers**. Think of sending a gift to a

friend in Pune: you write the card; the courier packs it with a tracking number; the hub writes the city and street; a van, a truck or a plane each moves it one stretch. Each person reads only their own label (the van driver never opens your card), and you can swap one part, a train instead of a truck, without changing the card.



Each person reads only their own label. Swap the truck for a train, and the card doesn't change.

Figure 2. The gift, layer by layer: the card is the application layer, the tracking number is transport, the address is the network layer, and the van is the link layer.

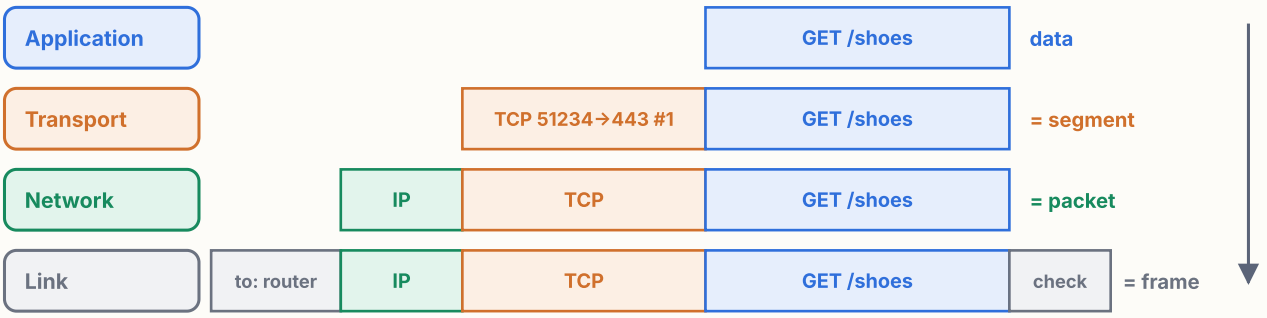
Layer	Protocols	Its one job	Unit
Application	HTTP, DNS, SMTP	what the app means	data
Transport	TCP, UDP	the right app (ports); for TCP, nothing lost, in order	segment
Network	IP	from and to addresses: find the device anywhere	packet
Link	Wi-Fi, Ethernet, mobile data	move the bits across one link, to the next device	frame

This four-layer version is the TCP/IP (internet) model. Lesson 6 covers the seven-layer OSI model you'll hear in interviews, and lesson 7 follows a URL through the TCP/IP model.

4 Step by step: encapsulation

1. **Application.** Your app writes the data: `GET /shoes`.
2. **Transport.** TCP adds its header in front: from port 51,234 to port 443, plus a sequence number. Data plus this header is a **segment**.
3. **Network.** IP adds its own header in front of that: from your IP address to the server's. Now it's a **packet**.
4. **Link.** The link layer wraps the whole packet in a **frame**: the hardware address of the next device (your Wi-Fi router) in front, and a check value at the end to catch damage.

This wrapping, layer by layer, is called **encapsulation**. At the other end it runs in reverse: each layer reads only its own header, removes it and passes the rest up, link, then IP, then TCP, until the web server reads `GET /shoes`. Like opening boxes inside boxes.

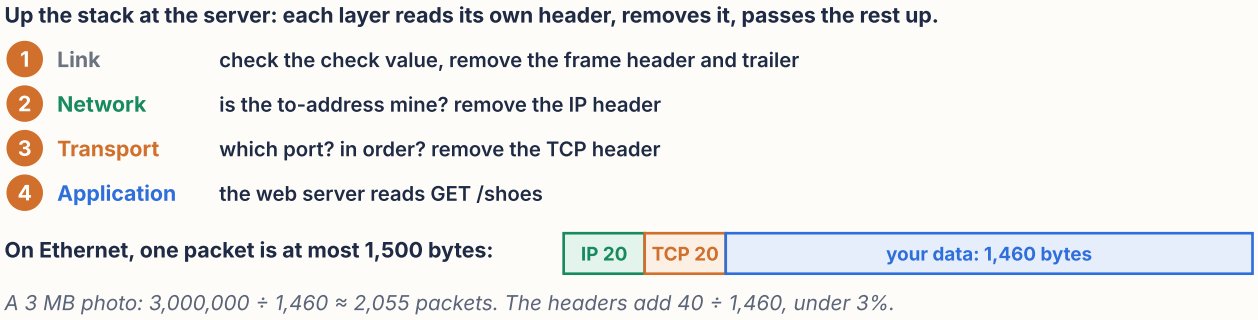


Encapsulation: each layer adds its own header in front (the link layer also adds a check value at the end).

Figure 3. Encapsulation on the way down: the data gets a TCP header (a segment), then an IP header (a packet), then a link header and trailer (a frame).

5 Real numbers

On Ethernet, one frame carries a packet of at most **1,500 bytes** (the MTU). The IP header usually takes 20 bytes and the TCP header at least 20, so each packet carries up to **1,460 bytes** of your data. A 3 MB photo is 3,000,000 bytes: 3,000,000 ÷ 1,460 ≈ **2,055 packets**, the photo in 2,000 pieces from lesson 2. All those headers add 40 ÷ 1,460, **under 3%**.



A 3 MB photo: 3,000,000 ÷ 1,460 ≈ 2,055 packets. The headers add 40 ÷ 1,460, under 3%.

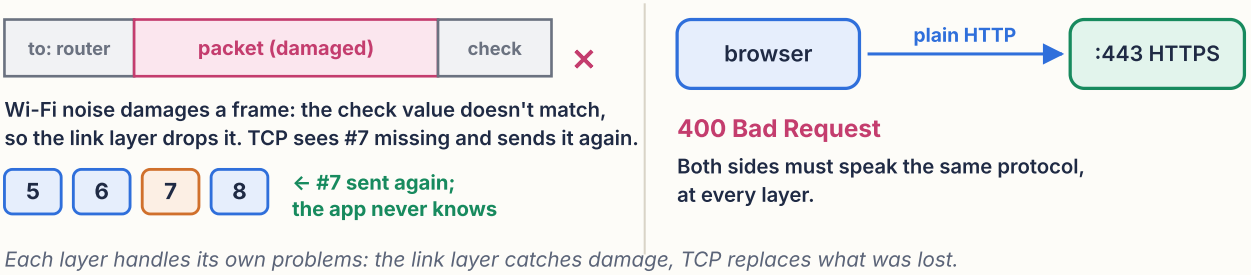
Figure 4. On the way up each layer removes only its own header. The sizes: 20 bytes of IP header and at least 20 of TCP leave up to 1,460 bytes of data in a 1,500-byte packet.

6 Why bother with layers

- **One job each.** Each layer does one thing and does it well.
- **Swap a layer.** Replace Wi-Fi with a cable, and HTTP, TCP and IP don't change at all.
- **Different builders.** The app maker, the operating system and the chip maker each build different layers, and they still fit together.

7 What goes wrong

- **A damaged frame.** Noise on the Wi-Fi damages a frame. The link layer checks the check value, sees it doesn't match, and drops the frame. The layer above notices: TCP sees a missing piece and sends it again (exactly how, in lesson 8). Your app never knows. Each layer handles its own problems.
- **The wrong protocol.** Send plain HTTP to a port that expects HTTPS, and the server can't read it: you get an error like `400 Bad Request`. Both sides must speak the same protocol, at every layer.



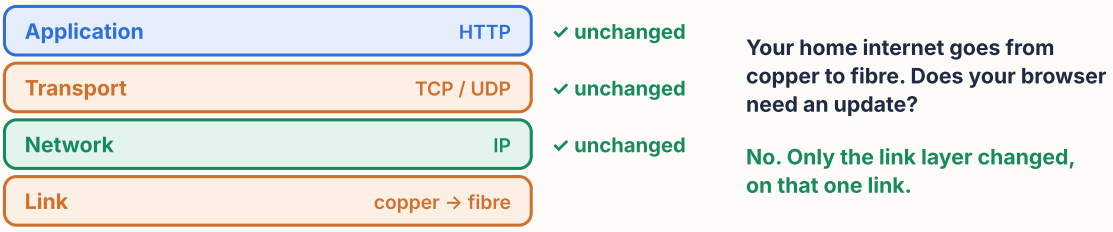
Each layer handles its own problems: the link layer catches damage, TCP replaces what was lost.

Figure 5. What goes wrong: a damaged frame is dropped by the link layer and resent by TCP, and a protocol mismatch (plain HTTP to an HTTPS port) gets an error.

PAUSE AND THINK

Your home internet is upgraded, from an old copper line to fibre. Does your browser need an update? Cover the answer and decide.

Answer. No. Only the bottom layer changed, on that one link. The network, transport and application layers above it work exactly as before. That's the whole point of layers.



That's the whole point of layers: change one without touching the others.

Figure 6. The pause-and-think answer: swapping copper for fibre changes only the link layer; the network, transport and application layers work exactly as before.

TWO COMMON MISTAKES

- **"TCP/IP is one protocol."** No. It's a family named after its two main members: TCP at the transport layer and IP at the network layer, with HTTP above and Wi-Fi or Ethernet below.
- **"Every device reads every header."** No. A router in the middle looks at the link and IP headers to pass the packet on; only the two end devices read the TCP header and the data.

RECAP IN THREE LINES

1. A protocol is a set of agreed rules, so two machines understand each other.
2. The internet stacks four layers, application, transport, network and link, each with one job.
3. Going down, each layer adds its header (encapsulation); going up, each one removes it.

8 Interview questions: what to say in 30 seconds

Q1. What is a protocol? Give an example. BASIC

A protocol is a set of agreed rules for how two sides exchange messages: what to send, in what format, in what order, and what to do when something goes wrong. HTTP is one: the client sends a request such as GET /shoes, and the server must answer with a status line such as 200 OK, headers and a body.

Q2. What are the layers of the internet (TCP/IP) model, and what does each do? BASIC

Four layers. Application (HTTP, DNS): what the app means. Transport (TCP, UDP): which app, by port, and for TCP, reliable in-order delivery. Network (IP): addressing and routing to the right device anywhere. Link (Ethernet, Wi-Fi): moving frames across one physical link to the next device.

Q3. What is encapsulation? BASIC

On the way down, each layer wraps what it gets from the layer above with its own header: data plus a TCP header is a segment, plus an IP header is a packet, plus a link header and trailer is a frame. On the way up, each layer reads and removes only its own header and passes the rest up.

Q4. Why do we use layers at all, instead of one big protocol? SCENARIO

Separation of concerns. Each layer does one job and only talks to the layers next to it, so you can change one without touching the others: replace Wi-Fi with fibre, or HTTP/1.1 with HTTP/2, and the rest keeps working. Different vendors can build different layers, and each layer handles its own errors.

Q5. How much of a 1,500-byte packet is your data? TRICKY

With a 20-byte IPv4 header and a 20-byte TCP header (no options), 1,460 bytes, which is the usual TCP maximum segment size on Ethernet. Options (timestamps are common) take 12 more, and IPv6 has a 40-byte header, so the real figure is often a bit less.

Q6. A frame is damaged on Wi-Fi. Which layer notices, and which layer fixes it? TRICKY

The link layer notices: the check value at the end of the frame doesn't match, so it drops the frame. It does not fix it; TCP, at the transport layer, sees the missing data and sends it again. The application never knows. Each layer handles the problems of its own job.

Read more. RFC 1122 (requirements for internet hosts: the layers), RFC 9110 (HTTP semantics), RFC 9293 (TCP), RFC 791 (IPv4), RFC 894 (IP over Ethernet, the 1,500-byte MTU), RFC 879 (TCP maximum segment size). The full written lesson is at backendroadmap.com.

Next: SD-N-006 · The OSI model, made simple backendroadmap.com